Atividade 3 - Nome: Henrique Almeida Pinho

1. script.js ->

```
async function cadastrarItem(nome) {
    const response = await fetch('http://localhost:3000/items', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name: nome })
    });
    const data = await response.json();
    return data;
}
```

2.

- **Prisma (schema.prisma):**

  Code snippet

```
model Task {
  id    Int     @id @default(autoincrement())
  title String
  done  Boolean @default(false)
}
```

- **Mongoose (JavaScript):**

  JavaScript

```
const TaskSchema = new mongoose.Schema({
  title: { type: String, required: true },
  done: { type: Boolean, default: false }
});
```

3.

```
app.put('/tasks/:id', async (req, res) => {
    const { id } = req.params;
    const taskAtualizada = await Task.findByIdAndUpdate(id, req.body, { new: true
    res.json(taskAtualizada);
});
```

4.

```
// Tentativa de duplicata
try {
  await User.create({ email: "teste@fatec.sp.gov.br" });
} catch (error) {
  console.log(error.code); // Exibirá 11000
  console.log("Erro: Este e-mail já está cadastrado!");
}
```

5.

Ferramenta	Sigla	Tipo de Banco	Exemplo de Banco	Estrutura de Dados
Prisma	ORM	Relacional (SQL)	PostgreSQL, MySQL, SQLite	Tabelas e Linhas
Mongoose	ODM	Documento (NoSQL)	MongoDB	Coleções e Documentos (JSON)

6. DELETE (Método HTTP para deletar um registro).
7. Task.find({}) (Comando Mongoose para buscar todos os documentos).
8. PATCH (Método HTTP para atualizar apenas um campo específico).
9. Mapear e persistir dados (Principal função do Mongoose entre a aplicação e o MongoDB).
10. DELETE (Método usado para a rota de exclusão no controle de estoque).
11. GET (Método usado para a rota que retorna todos os itens).
12. { type: String, required: true } (Configuração de campo obrigatório no Mongoose).
13. Alternativa B (Modelo Prisma com `id Int @id @default(autoincrement())`).

14. PATCH (Método ideal para alterar apenas o telefone de um cliente).
15. D) shoppingitems (Nome da coleção gerada pelo Prisma no MongoDB).
16. C) npx prisma studio (Comando para abrir a interface visual de dados).
17. C) Executar arquivos .ts diretamente no Node.js (Função da ferramenta `ts-node`).
18. C) O campo é opcional (pode ser nulo) (Significado do símbolo `?` no arquivo schema.prisma).
19. D) .env (Arquivo onde deve ser configurada a variável `DATABASE_URL`).
20. C) Erro de restrição única (Unique constraint) (Indicação do código de erro P2002 no Prisma).
21. C) Tentativa de inserir um valor que já existe em um campo @unique (Explicação prática para o erro P2002).
22. npx prisma db pull (Comando usado para realizar o Introspection, ou seja, ler um banco já existente).
23. npx prisma migrate dev (Comando que cria as tabelas no banco a partir do arquivo `schema.prisma`).
24. Introspection (O nome do processo de gerar o modelo Prisma a partir de um banco de dados atual).
25. @relation (Atributo utilizado no Prisma para configurar o vínculo entre duas tabelas).
26. 1:N (Um para Muitos) (Tipo de relacionamento onde, por exemplo, uma Categoria possui vários Produtos).
27. npx prisma generate (Comando que deve ser executado sempre que o schema muda para atualizar o "Prisma Client" no código).
28. Foreign Key (Chave Estrangeira) (Campo na tabela "Muitos" que armazena o ID da tabela "Um" para criar a conexão).
29. onDelete: Cascade (Configuração que faz com que, ao deletar um "Pai" - ex: Categoria -, todos os "Filhos" - ex: Produtos - sejam deletados automaticamente).
30. type: mongoose.Schema.Types.ObjectId (Forma como definimos uma referência a outro documento no Mongoose/MongoDB).
31. Transações ($transaction) (Recurso usado para garantir que um conjunto de operações seja executado totalmente ou cancelado se uma delas falhar).
32. Status 201 (Código HTTP retornado quando um registro é criado com sucesso).
33. Status 404 (Código HTTP retornado quando um registro (ID) não é encontrado no banco).
34. req.params (Objeto do Express onde pegamos o ID enviado na URL, como em `/produtos/:id`).
35. Try/Catch (Bloco de código essencial para capturar erros de conexão ou de validação do banco de dados e evitar que o servidor caia).

Exemplo Prático de Relacionamento (Questão 25/26)

```
model User {
  id    Int    @id @default(autoincrement())
  posts Post[] // Um usuário tem muitos posts
}

model Post {
  id       Int    @id @default(autoincrement())
  title    String
  author   User   @relation(fields: [authorId], references: [id])
  authorId Int    // Chave estrangeira
}
```